

Hyperion 2026 Plan (Condensed)

Please note: This is a streamlined version of our full planning document. Some details have been shortened or summarised — for the complete breakdown, refer to the original document.

Hardware

Problems identified in 2025

The hardware problems that were identified in 2025 are:

- Wheels didn't have enough grip when robots ran at high speeds (surging) therefore, slid on the field.
- Light sensor board was positioned in a way that it could have shorted when against the aluminium plates, and it was too low to the ground, where it scraped the painted white line at States.
- Mirror was unreliable due to the reflective foil, leading to unwanted reflections
- Robot relied on momentum and couldn't score from a distance, resulting in wasted battery when surging and missed scores due to overshooting.
- Robot could not always keep possession of the ball and had to rely on angling the robot while moving to guide the ball towards the goal

Background research

Wheels – (Luke)

Table () compares the previously used single Omnidirectional wheel to a new Double Omnidirectional wheel.

Key factors	Type of Wheel	
	Single Omni-Directional Wheel	Double Omni-Directional Wheel
Load Capacity	Lower because rollers handle more stress individually, reducing overall load capacity.	Higher because the load is distributed across more rollers, allowing greater support
Traction & Stability	Offers less traction, which can lead to slipping on smooth surfaces.	Provides greater traction and smoother, more controlled movement.
Size	Narrow and compact, suitable for smaller robots or limited spaces.	Wider area, offering better balance and surface contact.

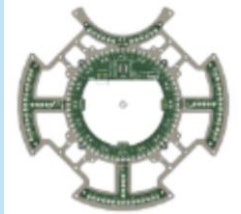
<i>Durability</i>	Rollers wear out faster due to concentrated stress.	Increased durability because the rollers share the load and wear more evenly.
<i>Maintenance</i>	More affordable but may require frequent roller replacement.	Higher initial cost but lower maintenance frequency over time.

The double omni wheel provides extra grip and stability to the robot while having higher durability for long use.

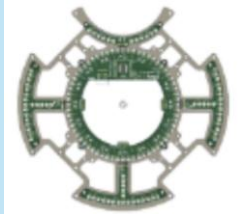
Out Avoidance – (Luke)

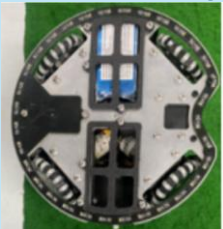
Due to the larger wheels, table () compares different shapes of light sensor boards to find the best solution for our problem.

Key Factors



Shape of Board

Quad perimeter sections and inner ring


Outside Wheel Ring


Inside wheel Ring


Mirror – (Luke)

Capture system – (Matthew)

The capture zone

Requirements/Restrictions - (Matthew)

Multiple requirements have to be considered in the design process. These requirements are:

Solutions – (Matthew)

Electrical

Problems identified in 2025

The electrical problems that were identified in 2025 are:

- Motor controllers were not easily accessible due to camera
- Camera and Teensy ports were inaccessible due to Bluetooth module
- TSSP's took too many pins on 1 Teensy 4.1
- LSB routing not optimal
- International communication module not considered in design before ordering PCBs & got reset when logic line switched off
- TSSPs gave inaccurate strength readings
- Robot positioning inaccurate due to mirror/camera limitations

Background research

Positioning on the field

Based on past international teams, ultrasonic sensors were used to detect the walls for positioning.

Model	Min/Max Distance	Resolution	Accuracy	Connection Method	Refresh rate	Cost (AUD)
DRFRobot-URM09-Ultrasonic-Distance Sensor	2cm – 500cm	1cm	1%	I ² C	20ms	
HC-SR04-Ultrasonic-Module-Distance-Measuring-Sensor	2cm – 400cm	3mm	3mm	Digital	50 – 100ms	

Requirements/Restrictions

Multiple requirements must be considered in the design process. These requirements are:

Solutions

Software

Problem Breakdown

The robot has multiple components incorporated into its design. To combine these components and their uses, an extensive code must be utilised.

Physical Components of our code include Bluetooth, Camera (Teensy & OpenMV Side), Drive System, Light System, TSSP System, Voltage Divider, Compass, Kicker (Mechanics & Strategy), Dribbler (Mechanics & Strategy).

Other Components of our code include the Movement Strategy (Defender & Attacker), PID, and Timers.

These must all be intricately connected in our main.cpp, and must have their respective libraries.

Background Research

A variety of background research was conducted, as can be seen in the following appendices:

1. TSSP System

The ball's direction can be detected using either vision or infrared systems. A vision-based system like the OpenMV H7 camera can identify the orange ball and coloured goals to estimate position but may lose sight if the ball is obscured. Alternatively, TSSP sensors can detect the ball's infrared light, offering more reliable and consistent tracking even with visual obstructions.

2. Attack Movement Strategy

Background research for this can be seen in Table 1.1 (Appendix).

3. Defender Movement Strategy

Background research for this can be seen in Table 1.2 (Appendix).

4. Light System

To stay within the field, a robot must detect the white boundary lines, which can be done using light sensors like phototransistors paired with LEDs. These sensors measure reflected light intensity, with white surfaces reflecting more than darker ones. Arranging the sensors and LEDs in a ring around the robot provides 360° coverage, allowing it to detect the line's presence and direction for corrective movement. This can be achieved by using the clustering method this allows for groups/clusters of sensor to be marked as detecti

Commented [SG1]: Ask tom to fix this up and add more stuff researching what type of line detecting methods can be used

ng the line, this allows for the centre of the cluster to be converted into a direction.

5. Camera System

OpenMV software uses Python to process images from an OpenMV camera, identifying the ball and goals by color through methods like blob tracking. Data is sent via UART in this order: two start packets, then X and Y coordinates for the yellow goal, blue goal, and ball. The receiving system will use serial functions to read these values and applies equations to calculate each object's angle and distance:

$$\text{Angle: } f(x, y) = (90 - \tan^{-1} \frac{y}{x}) \bmod 360$$
$$\text{Distance: } f(x, y) = \sqrt{x^2 + y^2}$$

6. Bluetooth System

Background Research for this can be seen in Table 1.3 (Appendix).

7. Kicker System

To use the kicker effectively, it will only activate when it aids in scoring. A timer prevents excessive firing, and a voltage check ensures sufficient charge before activation. The kicker is triggered by writing HIGH to a digital pin but only when the robot has ball possession (detected via a light gate), is aimed at the goal, and is on the opponent's side of the field.

8. Dribbler System

The robot maintains ball control by detecting when the ball enters its capture zone using ball strength sensors and activating a motor to pull it in. A simple state-based system manages this, keeping the motor off while idle and spinning it forward when the ball is detected, with timing control, current monitoring, and signal filtering improving reliability. For gameplay, the robot can use ball snatching, which is quick but risks losing control, or orbit and dribble, which provides smoother, more accurate positioning and strategic shooting, though slightly slower.

Specification of Requirements

In order for our robot's software to be 'successful', the following criterion must be met.

1. Ball Detection and Tracking
 - a. Accurately detect the orange ball using TSSP sensors (Lightweight) or the OpenMV Camera (Open)- (or both).
 - b. Track ball position in real-time and calculate angle and distance relative to the robot.
 - c. Maintain ball possession using the dribbler system.
2. Movement and Navigation
 - a. Implement attack and defender strategies according to the game state.

- b. Use motor movement calculations to ensure precise movement and smooth direction changes.
 - c. Detect and respond to white boundary lines using the light sensor system to stay inside the field.
 - d. Must be able to score from each neutral point on the field.
- 3. Goal Detection
 - a. Use OpenMV camera to detect yellow and blue goals.
 - b. Calculate angle and distance to opponent's goal for accurate targeting.
- 4. Kicker System
 - a. Only activate the kicker is in possession of the ball, aimed at the opponent's goal, and on the opponent's side of the field.
 - b. Implement a timer and voltage check to prevent premature or excessive kicking.
- 5. Dribbler System
 - a. Activate dribbler motor when the ball enters the capture zone.
 - b. Aide in the possession and accuracy of goal scoring.
- 6. Communication
 - a. Send and receive data via Bluetooth for debugging or external control.
 - b. Receive position data from the OpenMV camera via UART in the correct packet format.
- 7. System Integration
 - a. Ensure all components (camera, TSSP, dribbler, kicker, light sensors, PID, timers, motors) operate in coordination.
 - b. Maintain real-time performance for gameplay decisions.
- 8. Reliability Requirements
 - a. All software routines must handle unexpected data safely (e.g. Noise, Division by Zero in angle calculations).

Brainstorm, Evaluate and Choose Solutions

Ball Detection:

- Options: TSSP sensors only, OpenMV camera only, or hybrid system.
- Camera: Accurate angle/distance and goal detection but can fail if ball is obscured.
- TSSP: Lightweight, reliable infrared detection, works even if ball is partially hidden.
- **Chosen solution:** TSSP's (Lightweight), Camera (Open)

Movement Strategy - Attack

- Options: Cases, Exponential/Quadratic, PID

- **Chosen solution:** Exponential/Quadratic as it has a smooth curvature, medium processing demand, stable and predictable.

Boundary Detection

Commented [SG2]: Ask tom to do this

Dribbler & Kicker Activation

- The dribbler and kicker will be used to maintain possession, only being triggered when a certain amount of power is available, and when the ball is close enough.
- Adds timers, current monitoring, and signal filtering for reliability.

Communication

- Bluetooth will be used for switching communication and advanced strategy.
- UART to receive camera position data in correct packet format.

Integration & Final Solution

- All components integrated via **main.cpp**: camera, TSSP, dribbler, kicker, light sensors, PID, timers, and motors.
- Software loop:
 - Read sensors → calculate angle/distance → decide strategy → control motors & actuators.

Ultimate Robot Function:

- Detects and tracks the ball reliably.
- Maintains possession using dribbler.
- Navigates field safely and avoids boundaries.
- Scores efficiently with the kicker.
- Adapts between attacking and defending strategies.
- Operates reliably in real-time under game conditions.

Appendix

Orbit Type	What is it?	Positives	Negatives
Cases Orbit	A case's orbit is a set movement angle when the ball angle is within a certain range of angles.	• Easy to set up. • Predictable movement (always behave the same within each case). • Low processing demand.	• Lengthy and tedious tuning. • Jerky movement. • Does not adapt well to unexpected ball positions or fast movement. • Hard to maintain smooth transitions between cases.
Fixed Offset Orbit	A fixed offset orbit is a set constant that adds or subtracts from the ball's direction.	• Easy to set up. • Easy to tune. • Consistent and simple. • Low processing demand.	• Orbit can be very wide or too close at times, with no variance. • Lack's flexibility • Struggles when the ball moves quickly or unpredictably.
Exponential Polynomial Orbit	An exponential/polynomi al orbit is a constantly changing offset value that is subtracted or added to the ball's direction.	• Smooth movement. • Tightens naturally near the ball. • Looks natural and efficient in play-like situations. • Handles varied speed sand angles better than fixed systems.	• Complex to set up. • Harder to predict behaviour/interpret • Medium processing demand.
PID Orbit	A PID Orbit has a vertical and horizontal vector; it uses these vectors to wrap around the ball.	• Very smooth and adaptive movement around the ball • Continuously fixes itself based on readings; maintains stability. • Adjustable responsiveness. It can be very aggressive if needed.	• Very complex to set up. • Very complex to tune. • Can overcorrect or oscillate if not tuned properly. • Very high processing demand. • Relies on ball strength and distance; cannot function without.

Table 1.1 – Orbit Compare and Contrast

Table 1.2 – Defence Strategy Compare and Contrast

Method	What is it?	Positives	Negatives
Distance Constrained Defence	The robot orbits around the ball but stops moving forward once it reaches a set distance from the goal.	• Not commonly seen and provides a level of difference • Higher chance of scoring compared to staying in deep defence • Helps maintain goal positioning between the ball and goal.	• If opponents have a strong kicker, the robot may not orbit back fast enough to block the ball. • Can leave the goal open if timing or distance calibration is off.
Reactive Zone Defence	The robot stays between the goal and the centre of the field, moving or orbiting only when the ball enters its defensive zone.	• Conserves energy and reduces unnecessary movement. • Provides reliable defensive coverage when the ball approaches. • Easier to tune than complex systems.	• May take too long to react to fast-moving balls. • Can struggle against opponents with quick gameplay.
Floating Defender using Vectors	The robot keeps a constant distance from both the ball and the goal, always positioning itself between them to be ready for quick attacks or strong defence.	• Offers balanced positioning for both offence and defence. • Enables faster transitions between attacking and defending. • Maintains optimal alignment with the ball and goal.	• Requires precise vector calculations and tuning. • Can become unstable if the ball moves unpredictably and it is not tuned correctly.
Defender using Vectors	The robot moves sideways with the ball while saying a set distance away from the goal.	• Simple and stable defensive approach • Effective for cutting off shots to the side of the goal with goal tracking. • Easier to implement and maintain.	• Less responsive to central attacks. • Doesn't adapt to ball position relative to the goal (distance wise). • Can create open angles for shots if not tuned properly.

Table 1.3 – Bluetooth Strategy Compare and Contrast

Method	What is it?	Positives	Negatives	Combine with
Strongest Ball Strength	The robot with the strongest ball strength becomes the attacker. The other becomes the defender by default.	- Simple and reliable indicator of which robot is closest to the ball. - Fast decision-making with minimal data processing - Reduces confusion in determining possession. - Works without more complex components (e.g. Camera).	- Can cause rapid switching if both robots have similar ball readings. - Doesn't consider field position (may send both robots too far forward). - Could result in collisions near the ball if not paired with direction/goal checks.	Ball Direction Attack Cone. Distance-from-own-goal Fallback
Ball Direction Attack Cone	If a robot's shows the ball within a forward cone (e.g. ± 30 degrees of its heading) and the ball strength is ≥ 0.1 that of the other robot, the robot should be the attacker. The other robot becomes the defender by default.	- Ensures the attacker is properly oriented toward the ball. - Reduces risk of sideways or backwards approaches. - Promotes smoother, more strategic ball control and chasing behaviour. - Avoids conflicts where both robots see the ball from opposite sides.	- Relies on more accurate heading readings, errors can cause false alignment. - May ignore a closer robot if its angle is slightly off. - Slower to decide if angle calculations are noisy.	Strongest Ball Strength Goal Alignment
Goal Alignment	If a robot can see the opponent goal and the goal heading is generally forward from it, prefer that robot as attacker. The other becomes the defender by default.	- Encourages intelligent play by choosing robots that are well-positioned to score. - Prevents attackers moving in the wrong direction. - Improves orientation consistency.	- Goal detection can be inconsistent under lighting changes. - May fail if the goal is blocked or temporarily useless. - Doesn't help much during defence phases or when ball is near your own goal.	Ball Direction Attack Cone Distance-from-own-goal Fallback

		Increases scoring efficiency once ball possession is achieved.		
Distance-from-own-goal Fallback	Whichever robot is closer to the goal defaults to the defender. If both robots are near the defending goal, use other conditions (such as strongest ball strength) to determine roles.	- Ensures at least one robot is always defending the goal. - Prevents both robots from rushing too far up the field. - Provides structure and balanced team positioning. - Easy to determine using compass and estimated positioning using camera.	- Position may be slightly inaccurate/inconsistent at times. - May cause excessive hesitation if both robots stay defensive. - Can conflict with goal alignment if the "defender" also has a strong shot.	Strongest Ball Strength Goal Alignment
Battery Priority	If one robot's battery/temperature/health is below a safe threshold, deprioritize it for attacking tasks.	- Reduces risk of inconsistent sensor values or slowed gameplay on one robot compared to the other. - Reduces heat buildup and motor strain on weaker robot. - Prevents performance drops during key moments and scoring opportunities.	- Requires constant battery monitoring and sending this via Bluetooth. - Can cause uneven role assignments if one robot's battery remains low for the entire match. - Doesn't consider game context (e.g. what is happening in the game)	Strongest Ball Strength Distance-from-own-goal Fallback